Portfolio Reflection 2

Toby Pike

Product Showcase Website (Next.js + Contentful CMS)

For this reflection, I have chosen my Product Showcase Website project. This was a full-stack web application built using Next.js (React framework), Tailwind CSS, Contentful CMS, and role-based authentication logic. The purpose of the project was to create a dynamic product listing website where products are managed through a headless CMS and rendered dynamically on the frontend.

It includes:

- A homepage displaying products retrieved from Contentful
- Dynamic product detail pages using /products/[id] routing
- An authentication system using a custom AuthContext
- Role-based access control (RBAC) for protected features
- Environment variables to protect API keys
- A clean folder structure with separation of concerns

The project demonstrates both frontend and backend integration, API consumption, state management, authentication handling, and deployment preparation.

A link to a compressed folder of the project can be provided alongside this reflection.

What about the Process Worked Well

One of the biggest strengths of this project was the approach of structure and planning. Rather than building everything at once, the system was broken into clear components:

1. CMS setup (Contentful models and API access)
2. Next.js page structure and routing
3. Data fetching with environment variables
4. Authentication context
5. Protected UI components

This modular approach worked very well. It allowed me to isolate problems and debug more efficiently. Using Contentful proved to be a strong decision. It forced me to work with real-world API data instead of hardcoded mock data. It made the project feel much more realistic and closer to what I would experience in a production environment. Version control and environment configuration were also handled more professionally than my earlier projects. I properly used .env.local for API keys and ensured sensitive data was not exposed in my repository.

What about the Process Didn't Work Well

The biggest challenge was authentication and role-based access control. Managing user state in React, especially across page refreshes, required a deeper understanding of context and lifecycle than I initially had. I also underestimated how strict Next.js routing conventions are. Misplacing files inside the pages directory or incorrectly structuring dynamic routes caused errors that took time for me to diagnose. Another big learning moment for me was debugging environment variable issues. However, all these challenges were productive for my learning because they forced me to slow down and truly understand how the framework works rather than just copying patterns.

What About the Product Works Well

The finished product works well in several important ways:

Dynamic Rendering: Products are fetched from Contentful and rendered dynamically.

Scalable Routing: Each product has its own dynamic route.

Separation of Concerns: Data logic, UI components, and authentication logic are cleanly separated.

Role-Based Protection: Certain features are only available to authenticated or admin users.

Clean UI: Tailwind CSS provides a responsive, modern interface.

The structure of the application is clean and maintainable. It reflects how a real-world React/Next.js project should be organized.

What Could Be Improved:

If I were to improve this project further, I would:

- Implement a real authentication backend instead of simulated role logic.
- Improve error handling for API failures.

- Fully deploy it with production-ready configuration

While I think project is functional and structured well, adding these improvements would make it even better to prospective employers.

What I learned:

This project significantly deepened my understanding of:

Framework-Based Development:

Before this, I understood React, but Next.js forced me to think in terms of structure and conventions. File-based routing, server-side rendering concepts, and environment configuration gave me a clearer understanding of modern web application architecture.

Headless CMS Integration:

Using Contentful taught me how modern applications separate content management from presentation. I learned how to define content models, query API endpoints, handle asynchronous data, and map structured JSON data into UI components.

Authentication & State Management:

Building the AuthContext system helped me understand React Context API, state persistence, conditional rendering and role-based UI logic. This was a turning point in understanding frontend application architecture and I now feel comfortable implementing it.

Environment Configuration & Security

Managing .env.local files and API keys reinforced secure development practices. I now better understand how not to expose credentials.

How this Demonstrates my Computer Programming and Analysis Skills

This project demonstrates that I meet the expectations of a graduating Computer Programmer Analyst in several ways:

Technical Skills

- JavaScript
- React and Next.js
- REST API integration

- JSON data parsing
- CMS integration
- Environment configuration
- Frontend architecture design
- Role-based access logic

Analytical Thinking

This project required breaking down requirements into modular components, debugging routing and data-fetching issues, diagnosing authentication state problems and designing scalable folder structures.

Program Learning Outcomes Alignment

This project showcases outcomes such as:

- Designing and implementing software solutions
- Integrating multiple technologies
- Applying security and configuration best practices
- Demonstrating professional development standards
- Using version control and structured project architecture

Essential Employability Skills:

This project showcases some essential employability skills such as:

- Problem solving
- Independent learning
- Technical documentation
- Clean code organization
- Attention to detail

This project would likely impress a prospective employer because it demonstrates full-stack awareness. It shows integration, structure, and real-world thinking.

Final Reflection:

I'm proud of this project because it reflects my personal growth as a programmer. Earlier in the program, my focus was mostly on getting assignments done correctly and on time. With this project, I started thinking like a developer building something real. It pushed me technically, especially with authentication and API integration. It also helped build my confidence. I now feel capable of reading documentation, debugging framework issues, and integrating third-party services without feeling out of my depth. It shows that I can build structured, maintainable systems using multiple technologies together.